

01 How to find the version

```
In [39]: import pandas as pd  
pd.__version__
```

```
Out[39]: '2.0.2'
```

Another way to find version

```
In [40]: pd.show_versions()
```

INSTALLED VERSIONS

```

-----
commit           : 965ceca9fd796940050d6fc817707bba1c4f9bfff
python           : 3.11.4.final.0
python-bits      : 64
OS               : Windows
OS-release       : 10
Version          : 10.0.19045
machine          : AMD64
processor         : Intel64 Family 6 Model 78 Stepping 3, GenuineIntel
byteorder        : little
LC_ALL           : None
LANG             : None
LOCALE           : English_United States.1252

pandas           : 2.0.2
numpy            : 1.25.0
pytz             : 2023.3
dateutil         : 2.8.2
setuptools       : 65.5.0
pip              : 23.2
Cython           : None
pytest           : None
hypothesis       : None
sphinx           : None
blosc            : None
feather          : None
xlsxwriter       : None
lxml.etree       : None
html5lib         : None
pymysql          : None
psycopg2         : None
jinja2           : 3.1.2
IPython          : 8.14.0
pandas_datareader: None
bs4              : 4.12.2
bottleneck       : None
brotli           : None
fastparquet      : None
fsspec           : None
gcsfs            : None
matplotlib       : 3.7.1
numba            : None
numexpr          : None
odfpy            : None
openpyxl         : 3.1.2
pandas_gbq       : None
pyarrow          : None
pyreadstat       : None
pyxlsb           : None
s3fs             : None
scipy            : 1.10.1
snappy           : None
sqlalchemy       : None
tables           : None
tabulate         : None
xarray           : None
xlrd             : None
zstandard        : None
tzdata           : 2023.3

```

```
qtpy      : None
pyqt5     : None
```

02 Make Data Frame

```
In [41]: df=pd.DataFrame({'Name':['Ali','Hassan','Abdullah','Ahmad'], 'Age':[17,18,19,20]})
df
```

```
Out[41]:
```

	Name	Age
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

Make Data Frame using Numpy

```
In [42]: import numpy as np
arr=np.array([[1,2,3],[4,5,6],[7,8,9]])
pd.DataFrame(arr)
```

```
Out[42]:
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Another way

```
In [43]: #numpy array data frame
pd.DataFrame(np.random.rand(5,5))
```

```
Out[43]:
```

	0	1	2	3	4
0	0.722150	0.681265	0.725057	0.305084	0.976153
1	0.757917	0.583504	0.103849	0.035564	0.463114
2	0.534545	0.088026	0.113326	0.182462	0.059655
3	0.015898	0.708427	0.686193	0.363764	0.597501
4	0.370421	0.676361	0.229757	0.546515	0.111286

```
In [44]: # Set columns names
pd.DataFrame(np.random.rand(5,5), columns=list('ABCDE'))
```

Out[44]:

	A	B	C	D	E
0	0.648630	0.454158	0.962803	0.149249	0.989017
1	0.473225	0.263116	0.929591	0.978461	0.508162
2	0.206568	0.214384	0.797122	0.386934	0.704504
3	0.804269	0.685081	0.053828	0.611270	0.117708
4	0.776766	0.721382	0.986508	0.974661	0.629735

03 How to rename columns?

In [45]: *#Without rename*
 df=pd.DataFrame({'Name':['Ali','Hassan','Abdullah','Ahmad'], 'Age':[17,18,19,20]})
 df

Out[45]:

	Name	Age
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

In [46]: *#with rename*
 df.rename(columns={'Name':'Name1','Age':'Age1'}, inplace=True)
 df

Out[46]:

	Name1	Age1
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

Another way to rename column

In [47]: df.columns=['Name_1','Age_1']
 df

Out[47]:

	Name_1	Age_1
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20


```
In [48]: #replace any character, string
df.columns=df.columns.str.replace('_', ' ')
df
```

```
Out[48]:
```

	Name 1	Age 1
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

```
In [49]: df.columns=df.columns.str.replace(' ', '_')
df
```

```
Out[49]:
```

	Name_1	Age_1
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

```
In [50]: #adding prefix to columns
df.add_prefix('First_')
```

```
Out[50]:
```

	First_Name_1	First_Age_1
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

```
In [51]: #adding suffix to columns
df.add_suffix(' |')
```

```
Out[51]:
```

	Name_1 	Age_1
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

```
In [52]: #writting actual column names
df.columns=['Name1', 'Age1']
df
```

```
Out[52]:
```

	Name1	Age1
0	Ali	17
1	Hassan	18
2	Abdullah	19
3	Ahmad	20

4 Using template data

```
In [53]: import pandas as pd
import numpy as np
import seaborn as sns
```

```
In [54]: phool=sns.load_dataset('tips')
phool.head()
```

```
Out[54]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
In [55]: #Check information
phool.info()
```

```
#Check Statistical Summry
phool.describe()
```

```
#Check columns
phool.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null    float64
1   tip         244 non-null    float64
2   sex         244 non-null    category
3   smoker      244 non-null    category
4   day         244 non-null    category
5   time        244 non-null    category
6   size        244 non-null    int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
```

```
Out[55]: Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')
```

```
In [56]: #save dataset  
#phool.to_csv('tips.csv')  
phool.to_excel('tips.xlsx')
```

5 Using your own data

```
In [57]: import pandas as pd  
df=pd.read_csv("ml_data.csv")  
df
```

Out[57]:

	YearsExperience	Age	distance	Salary
0	1.1	21.0	77.75	39343
1	1.3	21.5	78.25	46205
2	1.5	21.7	78.25	37731
3	2.0	22.0	80.00	43525
4	2.2	22.2	80.00	39891
5	2.9	23.0	80.50	56642
6	3.0	23.0	82.50	60150
7	3.2	23.3	87.60	54445
8	3.2	23.3	87.60	64445
9	3.7	23.6	88.60	57189
10	3.9	23.9	87.70	63218
11	4.0	24.0	90.00	55794
12	4.0	24.0	9.00	56957
13	4.1	24.0	91.00	57081
14	4.5	25.0	93.00	61111
15	4.9	25.0	95.00	67938
16	5.1	26.0	97.00	66029
17	5.3	27.0	98.00	83088
18	5.9	28.0	98.00	81363
19	6.0	29.0	98.50	93940
20	6.8	30.0	98.40	91738
21	7.1	30.0	98.50	98273
22	7.9	31.0	98.70	101302
23	8.2	32.0	98.80	113812
24	8.7	33.0	98.90	109431
25	9.0	34.0	99.00	105582
26	9.5	35.0	99.20	116969
27	9.6	36.0	99.40	112635
28	10.3	37.0	99.50	122391
29	10.5	38.0	99.70	121872

6 Reverse Row Order

```
In [58]: import seaborn as sns
import pandas as pd
df=sns.load_dataset("tips")
df.head()
```

```
Out[58]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
In [59]: df.loc[::-1].head()
```

```
Out[59]:
```

	total_bill	tip	sex	smoker	day	time	size
243	18.78	3.00	Female	No	Thur	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
240	27.18	2.00	Female	Yes	Sat	Dinner	2
239	29.03	5.92	Male	No	Sat	Dinner	3

```
In [60]: #reset index
df.loc[:,1].reset_index(drop=True).head()
```

```
Out[60]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

7- Reverse column order

```
In [61]: df.loc[:,::-1].head()
```

```
Out[61]:
```

	size	time	day	smoker	sex	tip	total_bill
0	2	Dinner	Sun	No	Female	1.01	16.99
1	3	Dinner	Sun	No	Male	1.66	10.34
2	3	Dinner	Sun	No	Male	3.50	21.01
3	2	Dinner	Sun	No	Male	3.31	23.68
4	4	Dinner	Sun	No	Female	3.61	24.59

8- Select a column by dtype

```
In [62]: df.dtypes
```

```
Out[62]: total_bill    float64
tip              float64
sex              category
smoker           category
day             category
time            category
size            int64
dtype: object
```

```
In [63]: #only select those have numeric type
df.select_dtypes(include=['number']).head()
```

```
Out[63]:
```

	total_bill	tip	size
0	16.99	1.01	2
1	10.34	1.66	3
2	21.01	3.50	3
3	23.68	3.31	2
4	24.59	3.61	4

```
In [64]: #only select those have object type
df.select_dtypes(include=['object']).head()
```

```
Out[64]:
```

0
1
2
3
4

```
In [65]: #only select those have category type
df.select_dtypes(include=['category']).head()
```

Out[65]:

	sex	smoker	day	time
0	Female	No	Sun	Dinner
1	Male	No	Sun	Dinner
2	Male	No	Sun	Dinner
3	Male	No	Sun	Dinner
4	Female	No	Sun	Dinner

In [66]: *#select those have multiple type*
`df.select_dtypes(include=['category', 'number']).head()`

Out[66]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

In [67]: *#for excluding*
#only exclude those have number type
`df.select_dtypes(exclude=['number']).head()`

Out[67]:

	sex	smoker	day	time
0	Female	No	Sun	Dinner
1	Male	No	Sun	Dinner
2	Male	No	Sun	Dinner
3	Male	No	Sun	Dinner
4	Female	No	Sun	Dinner

9- Convert strings to numbers

In [68]: `df=pd.DataFrame({'col_A':['1','2','3','4','5','6'], 'col_B':['7','8','9','10','11','12']})`
`df`

```
Out[68]:
```

	col_A	col_B
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

```
In [69]: df.dtypes
```

```
Out[69]: col_A    object
col_B    object
dtype: object
```

```
In [70]: df.astype({'col_A':'int64','col_B':'int64'}).dtypes
```

```
Out[70]: col_A    int64
col_B    int64
dtype: object
```

10- Reduce DataFrame size

```
In [71]: df=sns.load_dataset("titanic")
df
```

```
Out[71]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	d
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	N
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	N
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	N
...	...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	N
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	N
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	N

891 rows × 15 columns

```
In [72]: df.sample(frac=0.5).shape
```


Out[72]: (446, 15)

11- Copy Data From Clipboard

```
In [73]: #Loading dataset
import pandas as pd
import seaborn as sns

df=sns.load_dataset("titanic")
#save dataset in excel format
df.to_excel("kashti.xlsx")
```

```
In [74]: #read clipboard data in python
df=pd.read_clipboard()
df
```

```
Out[74]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	22.0	1	0	7.25	S	Third
1	1	1	female	38.0	1	0	71.28	C	First
2	1	3	female	26.0	0	0	7.93	S	Third
3	1	1	female	35.0	1	0	53.10	S	First
4	0	3	male	35.0	0	0	8.05	S	Third
5	0	3	male	NaN	0	0	8.46	Q	Third
6	0	1	male	54.0	0	0	51.86	S	First
7	0	3	male	2.0	3	1	21.08	S	Third
8	1	3	female	27.0	0	2	11.13	S	Third
9	1	2	female	14.0	1	0	30.07	C	Second
10	1	3	female	4.0	1	1	16.70	S	Third
11	1	1	female	58.0	0	0	26.55	S	First
12	0	3	male	20.0	0	0	8.05	S	Third
13	0	3	male	39.0	1	5	31.28	S	Third
14	0	3	female	14.0	0	0	7.85	S	Third
15	1	2	female	55.0	0	0	16.00	S	Second

12- Split dataframe into two subsets

```
In [75]: #Loading dataset
import pandas as pd
import seaborn as sns

df=sns.load_dataset("titanic")
df.head()
```

Out[75]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

In [76]: `len(df)`

Out[76]: 891

In [77]: `df.shape`

Out[77]: (891, 15)

In [78]: `from random import random
kashti_1=df.sample(frac=0.5, random_state=1)
kashti_1.shape`

Out[78]: (446, 15)

In [79]: `kashti_2=df.drop(kashti_1.index)
kashti_2.shape`

Out[79]: (445, 15)

In [80]: `len(kashti_1) + len(kashti_2)`

Out[80]: 891

In [81]: `kashti_1.head()`

Out[81]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	d
862	1	1	female	48.0	0	0	25.9292	S	First	woman	False	
223	0	3	male	NaN	0	0	7.8958	S	Third	man	True	N
84	1	2	female	17.0	0	0	10.5000	S	Second	woman	False	N
680	0	3	female	NaN	0	0	8.1375	Q	Third	woman	False	N
535	1	2	female	7.0	0	2	26.2500	S	Second	child	False	N

In [82]: `kashti_2.head()`

Out[82]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	
7	0	3	male	2.0	3	1	21.0750	S	Third	child	False	Na
10	1	3	female	4.0	1	1	16.7000	S	Third	child	False	
15	1	2	female	55.0	0	0	16.0000	S	Second	woman	False	Na
18	0	3	female	31.0	1	0	18.0000	S	Third	woman	False	Na

In [83]: `df.shape`

Out[83]: (891, 15)

In [84]: `len(kashti_1) + len(kashti_2)`

Out[84]: 891

13- Join two dataframes

In [85]: `import pandas as pd`
`df1=pd.concat([kashti_1,kashti_2])`
`df1.shape`

Out[85]: (891, 15)

14- Filtering a dataset

In [86]: `df.head()`

Out[86]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

In [87]: `df.sex.unique()`

Out[87]: array(['male', 'female'], dtype=object)

In [88]: `#filtering`
`df[(df.sex=="female")]`

Out[88]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	d
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	N
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	N
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	N
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	False	N
9	1	2	female	14.0	1	0	30.0708	C	Second	child	False	N
...	...	...	...	...	...	...	...	...	...	...	...	...
880	1	2	female	25.0	0	1	26.0000	S	Second	woman	False	N
882	0	3	female	22.0	0	0	10.5167	S	Third	woman	False	N
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	False	N
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	N
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	N

314 rows × 15 columns

In [89]: *#another example*
df.embark_town.unique()

Out[89]: array(['Southampton', 'Cherbourg', 'Queenstown', nan], dtype=object)

In [90]: df[(df.embark_town=='Southampton')].shape

Out[90]: (644, 15)

In [91]: df[((df.embark_town=='Southampton') |
(df.embark_town=='Queenstown')) &
(df.sex=='female')
].shape

Out[91]: (239, 15)

In [92]: *#another way*
df[(df.embark_town.isin(['Southampton', 'Queenstown']))].head()

Out[92]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN
5	0	3	male	NaN	0	0	8.4583	Q	Third	man	True	NaN

In [93]: df[df.age<30].shape

Out[93]: (384, 15)

15- Filtering by large categories

In [94]: `df.age.value_counts()`

Out[94]:

```
age
24.00    30
22.00    27
18.00    26
19.00    25
28.00    25
..
36.50     1
55.50     1
0.92      1
23.50     1
74.00     1
Name: count, Length: 88, dtype: int64
```

In [95]: `count=df.age.value_counts()`
`count.nlargest(2)`

Out[95]:

```
age
24.0    30
22.0    27
Name: count, dtype: int64
```

In [96]: `count=df.age.value_counts()`
`count.nlargest(2).index`

Out[96]: Index([24.0, 22.0], dtype='float64', name='age')

In [97]: *#another example*
`count=df.who.value_counts()`
`count.nlargest(3)`

Out[97]:

```
who
man      537
woman    271
child     83
Name: count, dtype: int64
```

In [98]: *#another way*
`df[df.who.isin(count.nlargest(1).index)].head()`

Out[98]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	en
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Sc
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Sc
5	0	3	male	NaN	0	0	8.4583	Q	Third	man	True	NaN	C
6	0	1	male	54.0	0	0	51.8625	S	First	man	True	E	Sc
12	0	3	male	20.0	0	0	8.0500	S	Third	man	True	NaN	Sc

16- Splitting a string into multiple columns

```
In [99]: import pandas as pd
df=pd.DataFrame({'Name':["Rao Ali","Rao Hassan","Rao Abdullah","Rao Ahmad"],
                 'Location':["Lodhran,Pakistan","Bahawalpur,Pakistan","Multan,Pakistan","Lahore,Pakistan"]})
```

```
Out[99]:
```

	Name	Location
0	Rao Ali	Lodhran,Pakistan
1	Rao Hassan	Bahawalpur,Pakistan
2	Rao Abdullah	Multan,Pakistan
3	Rao Ahmad	Lahore,Pakistan

```
In [100... #splitting "Name" coloumn into two coloumns
df.Name.str.split(' ',expand=True)
```

```
Out[100]:
```

	0	1
0	Rao	Ali
1	Rao	Hassan
2	Rao	Abdullah
3	Rao	Ahmad

```
In [101... #Adding those splits into new coloumns
df[["first_name", "last_name"]]=df.Name.str.split(' ',expand=True)
df
```

```
Out[101]:
```

	Name	Location	first_name	last_name
0	Rao Ali	Lodhran,Pakistan	Rao	Ali
1	Rao Hassan	Bahawalpur,Pakistan	Rao	Hassan
2	Rao Abdullah	Multan,Pakistan	Rao	Abdullah
3	Rao Ahmad	Lahore,Pakistan	Rao	Ahmad

```
In [102... #Splitting "Location" coloumn into new columns
df.Location.str.split(',', expand=True)
```

```
Out[102]:
```

	0	1
0	Lodhran	Pakistan
1	Bahawalpur	Pakistan
2	Multan	Pakistan
3	Lahore	Pakistan

```
In [103... #Adding those columns into new columns
df[["City", "Country"]] = df.Location.str.split(',', expand=True)
df
```

```
Out[103]:
```

	Name	Location	first_name	last_name	City	Country
0	Rao Ali	Lodhran,Pakistan	Rao	Ali	Lodhran	Pakistan
1	Rao Hassan	Bahawalpur,Pakistan	Rao	Hassan	Bahawalpur	Pakistan
2	Rao Abdullah	Multan,Pakistan	Rao	Abdullah	Multan	Pakistan
3	Rao Ahmad	Lahore,Pakistan	Rao	Ahmad	Lahore	Pakistan

```
In [104... #Refine data manipulation
df=df[["first_name", "last_name", "City", "Country"]]
df
```

```
Out[104]:
```

	first_name	last_name	City	Country
0	Rao	Ali	Lodhran	Pakistan
1	Rao	Hassan	Bahawalpur	Pakistan
2	Rao	Abdullah	Multan	Pakistan
3	Rao	Ahmad	Lahore	Pakistan

17- Aggregate by multiple groups/functions

```
In [105... #import Libraries
import pandas as pd
import seaborn as sns

#importing dataset
df=sns.load_dataset('titanic')
df.head()
```

```
Out[105]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

```
In [106... df.groupby("who").count()
```

Out[106]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	adult_male	deck	embark_t
who												
child	83	83	83	83	83	83	83	83	83	83	13	
man	537	537	537	413	537	537	537	537	537	537	99	
woman	271	271	271	218	271	271	271	269	271	271	91	

In []: `len(df.groupby('sex'))`

Out[]: 2

In []: `df.groupby(["sex", "pclass", "embarked"]).count()`

Out[]:

		survived	age	sibsp	parch	fare	class	who	adult_male	deck	embark_t
sex	pclass	embarked									
female	1	C	43	38	43	43	43	43	43	35	
		Q	1	1	1	1	1	1	1	1	
		S	48	44	48	48	48	48	48	43	
	2	C	7	7	7	7	7	7	7	1	
		Q	2	1	2	2	2	2	2	1	
		S	67	66	67	67	67	67	67	8	
	3	C	23	16	23	23	23	23	23	1	
		Q	33	10	33	33	33	33	33	0	
		S	88	76	88	88	88	88	88	5	
male	1	C	42	36	42	42	42	42	42	31	
		Q	1	1	1	1	1	1	1	1	
		S	79	64	79	79	79	79	79	62	
	2	C	10	8	10	10	10	10	10	1	
		Q	1	1	1	1	1	1	1	0	
		S	97	90	97	97	97	97	97	5	
	3	C	43	25	43	43	43	43	43	0	
		Q	39	14	39	39	39	39	39	1	
		S	265	214	265	265	265	265	265	5	

18- Select specific rows or columns

In []: `df.head()`

Out[]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

In []: `#select specific columns`
`df[['pclass','sex','age','fare']]`

Out[]:

	pclass	sex	age	fare
0	3	male	22.0	7.2500
1	1	female	38.0	71.2833
2	3	female	26.0	7.9250
3	1	female	35.0	53.1000
4	3	male	35.0	8.0500
...	...	...	...	...
886	2	male	27.0	13.0000
887	1	female	19.0	30.0000
888	3	female	NaN	23.4500
889	1	male	26.0	30.0000
890	3	male	32.0	7.7500

891 rows × 4 columns

In []: `#by using describe function`
`df.describe()`

Out[]:

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [ ]: #selecting specific rows
df.describe().loc[['mean', 'std', 'min', '50%', 'max']]
```

```
Out[ ]:
```

	survived	pclass	age	sibsp	parch	fare
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [ ]: #another way for selecting specific rows
df.describe().loc['std':'75%']
```

```
Out[ ]:
```

	survived	pclass	age	sibsp	parch	fare
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000

```
In [ ]: #selecting specific rows with specific columns
df.describe().loc['std':'50%', 'age':'fare']
```

```
Out[ ]:
```

	age	sibsp	parch	fare
std	14.526497	1.102743	0.806057	49.693429
min	0.420000	0.000000	0.000000	0.000000
25%	20.125000	0.000000	0.000000	7.910400
50%	28.000000	0.000000	0.000000	14.454200

19- Select Multi-index series

```
In [ ]: df.head()
```

```
Out[ ]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

```
In [ ]: #survived mean of sex column
df.groupby('sex').survived.mean()
```

```
Out[ ]: sex
female    0.742038
male      0.188908
Name: survived, dtype: float64
```

```
In [ ]: #survived mean of sex and pclass column
df.groupby(['sex', 'pclass']).survived.mean()
```

```
Out[ ]: sex    pclass
female  1      0.968085
         2      0.921053
         3      0.500000
male    1      0.368852
         2      0.157407
         3      0.135447
Name: survived, dtype: float64
```

```
In [ ]: #unstack
df.groupby(['sex', 'pclass']).survived.mean().unstack()
```

```
Out[ ]: pclass    1      2      3
sex
female  0.968085  0.921053  0.500000
male    0.368852  0.157407  0.135447
```

20- Continous to catagorical data conversion

```
In [ ]: df.head()
```

```
Out[ ]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

```
In [ ]: df.age.head()
```

```
Out[ ]:
```

0	22.0
1	38.0
2	26.0
3	35.0
4	35.0

Name: age, dtype: float64

```
In [ ]: #creating bins
pd.cut(df.age,bins=[0,18,26,99], labels=['child','young_adult','adult']).head()
```

```
Out[ ]:
```

0	young_adult
1	adult
2	young_adult
3	adult
4	adult

Name: age, dtype: category
Categories (3, object): ['child' < 'young_adult' < 'adult']

```
In [ ]: #add new column of 'new_age'
df['new_age']=pd.cut(df.age,bins=[0,18,26,99], labels=['child','young_adult','adult'])
df.head()
```

```
Out[ ]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

21- Convert one set of values into anotherone

```
In [ ]: df.sex.head()
```

```
Out[ ]: 0    male
        1    female
        2    female
        3    female
        4    male
        Name: sex, dtype: object
```

```
In [ ]: df.sex.map({'male':1,'female':0})
```

```
Out[ ]: 0      1
        1      0
        2      0
        3      0
        4      1
        ..
       886      1
       887      0
       888      0
       889      1
       890      1
        Name: sex, Length: 891, dtype: int64
```

```
In [ ]: #adding new column of 'sex_num'
df['sex_num']=df.sex.map({'male':1, 'female':0})
df.head()
```

```
Out[ ]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

```
In [ ]: df.embarked.unique()
```

```
Out[ ]: array(['S', 'C', 'Q', nan], dtype=object)
```

```
In [ ]: #convert multiple values
df['embarked_num']=df.embarked.factorize()[0]
df.head()
```

```
Out[ ]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN

22- Transpose a wide dataframe

```
In [ ]: import numpy as np
import pandas as pd
```

```
In [ ]: #creating a new data frame
df=pd.DataFrame(np.random.rand(200,25), columns=list('abcdefghijklmnopqrstuvwxy'))
df.head(10)
```

```
Out[ ]:
```

	a	b	c	d	e	f	g	h	i	j
0	0.492804	0.601086	0.471521	0.891114	0.212443	0.034398	0.778977	0.749846	0.184347	0.789103
1	0.217950	0.022724	0.099850	0.392980	0.347169	0.383803	0.408407	0.345019	0.977727	0.630701
2	0.878129	0.165385	0.848146	0.078084	0.783331	0.574973	0.645214	0.740768	0.843286	0.995689
3	0.193893	0.801852	0.898544	0.180413	0.772039	0.447974	0.607098	0.707322	0.100776	0.654596
4	0.636152	0.651800	0.885679	0.349138	0.398490	0.939455	0.668785	0.771305	0.149543	0.294412
5	0.381491	0.193375	0.114289	0.126256	0.363125	0.046561	0.469371	0.784259	0.781872	0.701485
6	0.937381	0.550764	0.115742	0.314202	0.088297	0.526635	0.818489	0.327571	0.447121	0.971562
7	0.114495	0.924046	0.243892	0.201907	0.516907	0.345138	0.182094	0.680229	0.741187	0.577924
8	0.072186	0.810820	0.409266	0.978069	0.654390	0.981880	0.026202	0.526041	0.728359	0.688390
9	0.389817	0.856653	0.997385	0.011268	0.843947	0.613389	0.572491	0.480964	0.560898	0.316683

10 rows × 25 columns

```
In [ ]: #Transpose
df.head(10).T
```

Out[]:

	0	1	2	3	4	5	6	7	8	9
a	0.492804	0.217950	0.878129	0.193893	0.636152	0.381491	0.937381	0.114495	0.072186	0.389817
b	0.601086	0.022724	0.165385	0.801852	0.651800	0.193375	0.550764	0.924046	0.810820	0.856653
c	0.471521	0.099850	0.848146	0.898544	0.885679	0.114289	0.115742	0.243892	0.409266	0.997385
d	0.891114	0.392980	0.078084	0.180413	0.349138	0.126256	0.314202	0.201907	0.978069	0.011268
e	0.212443	0.347169	0.783331	0.772039	0.398490	0.363125	0.088297	0.516907	0.654390	0.843947
f	0.034398	0.383803	0.574973	0.447974	0.939455	0.046561	0.526635	0.345138	0.981880	0.613389
g	0.778977	0.408407	0.645214	0.607098	0.668785	0.469371	0.818489	0.182094	0.026202	0.572491
h	0.749846	0.345019	0.740768	0.707322	0.771305	0.784259	0.327571	0.680229	0.526041	0.480964
i	0.184347	0.977727	0.843286	0.100776	0.149543	0.781872	0.447121	0.741187	0.728359	0.560898
j	0.789103	0.630701	0.995689	0.654596	0.294412	0.701485	0.971562	0.577924	0.688390	0.316683
k	0.104926	0.684336	0.762491	0.602638	0.491915	0.310468	0.226392	0.135313	0.276059	0.675311
l	0.715773	0.651603	0.885419	0.813551	0.632754	0.598436	0.182722	0.544372	0.285856	0.512851
m	0.038797	0.893583	0.122032	0.564823	0.788829	0.066887	0.422686	0.169911	0.851618	0.953743
n	0.805901	0.255175	0.729854	0.324385	0.033772	0.252719	0.777731	0.369623	0.900533	0.283963
o	0.259490	0.415133	0.453786	0.869526	0.151610	0.642536	0.389253	0.343515	0.746724	0.819661
p	0.360307	0.810492	0.495484	0.312158	0.330348	0.036819	0.152171	0.275473	0.878181	0.662192
q	0.592035	0.758002	0.615223	0.237240	0.466567	0.601067	0.016769	0.010556	0.760932	0.709827
r	0.861945	0.405661	0.124962	0.981610	0.441373	0.370536	0.008591	0.639300	0.151103	0.954553
s	0.173249	0.781297	0.796824	0.705473	0.713905	0.994118	0.098182	0.482952	0.652180	0.971659
t	0.206668	0.570581	0.954488	0.558573	0.990569	0.624501	0.565639	0.061122	0.404848	0.297833
u	0.011305	0.305947	0.650136	0.638474	0.446593	0.250424	0.134554	0.476618	0.111980	0.828488
v	0.113701	0.272566	0.971244	0.697349	0.436028	0.135952	0.738923	0.120921	0.457959	0.559078
w	0.921090	0.422593	0.427266	0.383219	0.314397	0.462197	0.182464	0.925361	0.278741	0.401914
x	0.493964	0.311205	0.641386	0.467580	0.886810	0.599479	0.633229	0.157936	0.036504	0.649034
y	0.071583	0.273335	0.457034	0.246740	0.773108	0.482195	0.170098	0.932285	0.996635	0.229037

In []: `#without transpose`
`df.describe()`

Out[]:

	a	b	c	d	e	f	g	h
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000
mean	0.492837	0.519962	0.507040	0.510189	0.522337	0.464703	0.493988	0.489788
std	0.277748	0.271201	0.306488	0.293134	0.279976	0.287973	0.304354	0.275250
min	0.003432	0.015588	0.004589	0.011268	0.004802	0.017399	0.005014	0.001954
25%	0.262614	0.308351	0.211014	0.236988	0.272117	0.218439	0.197533	0.265890
50%	0.482800	0.536976	0.524556	0.532202	0.514748	0.460131	0.503902	0.495342
75%	0.721248	0.735247	0.775340	0.751148	0.773778	0.699882	0.758971	0.703937
max	0.995373	0.994420	0.999771	0.985178	0.994602	0.999358	0.998448	0.999829

8 rows × 25 columns



```
In [ ]: #with transpose
df.describe().T
```


Out[]:

	count	mean	std	min	25%	50%	75%	max
a	200.0	0.492837	0.277748	0.003432	0.262614	0.482800	0.721248	0.995373
b	200.0	0.519962	0.271201	0.015588	0.308351	0.536976	0.735247	0.994420
c	200.0	0.507040	0.306488	0.004589	0.211014	0.524556	0.775340	0.999771
d	200.0	0.510189	0.293134	0.011268	0.236988	0.532202	0.751148	0.985178
e	200.0	0.522337	0.279976	0.004802	0.272117	0.514748	0.773778	0.994602
f	200.0	0.464703	0.287973	0.017399	0.218439	0.460131	0.699882	0.999358
g	200.0	0.493988	0.304354	0.005014	0.197533	0.503902	0.758971	0.998448
h	200.0	0.489788	0.275250	0.001954	0.265890	0.495342	0.703931	0.999829
i	200.0	0.546933	0.284634	0.004045	0.314176	0.570786	0.781109	0.997800
j	200.0	0.459475	0.278037	0.001190	0.228081	0.424217	0.676404	0.995689
k	200.0	0.499999	0.292799	0.000531	0.246933	0.495875	0.755941	0.998845
l	200.0	0.480475	0.272418	0.002330	0.271512	0.487160	0.705799	0.998318
m	200.0	0.493477	0.300715	0.000053	0.182317	0.515375	0.754973	0.999498
n	200.0	0.485963	0.291914	0.003061	0.251072	0.485050	0.746007	0.976967
o	200.0	0.513595	0.277234	0.004877	0.281577	0.499150	0.741643	0.999619
p	200.0	0.497623	0.282339	0.008169	0.257973	0.484265	0.772327	0.999227
q	200.0	0.528182	0.299235	0.004553	0.236736	0.554884	0.791402	0.989556
r	200.0	0.526880	0.295222	0.004711	0.293771	0.546128	0.804632	0.997702
s	200.0	0.501351	0.273499	0.006252	0.262589	0.504229	0.743684	0.994118
t	200.0	0.514763	0.293337	0.002664	0.246266	0.516849	0.756050	0.993739
u	200.0	0.484766	0.312655	0.007927	0.199481	0.468829	0.800371	0.997415
v	200.0	0.490458	0.279820	0.005083	0.248108	0.514607	0.706319	0.998755
w	200.0	0.500862	0.272903	0.000905	0.286419	0.509068	0.715563	0.995375
x	200.0	0.475461	0.286732	0.000242	0.254401	0.486439	0.670346	0.992996
y	200.0	0.507847	0.291072	0.000260	0.269087	0.500123	0.740155	0.996635

23- Reshaping a Dataframe

In []:

```
#Creating Dataframe
fasla=pd.DataFrame([[ '12345',100,200,300],[ '34567',400,500,600],[ '67890',700,800,900]]
                    columns=['zip','factory','warehouse','retail'])
fasla.head()
```

```
Out[ ]:
```

	zip	factory	warehouse	retail
0	12345	100	200	300
1	34567	400	500	600
2	67890	700	800	900

```
In [ ]: fasla.head().T
```

```
Out[ ]:
```

	0	1	2
zip	12345	34567	67890
factory	100	400	700
warehouse	200	500	800
retail	300	600	900

```
In [ ]: #Creating new dataframe
fasla2=pd.DataFrame([[1,'12345','factory'],[2,'34567','warehouse']],
                    columns=['user_id','zip','location_type'])
fasla2.head()
```

```
Out[ ]:
```

	user_id	zip	location_type
0	1	12345	factory
1	2	34567	warehouse

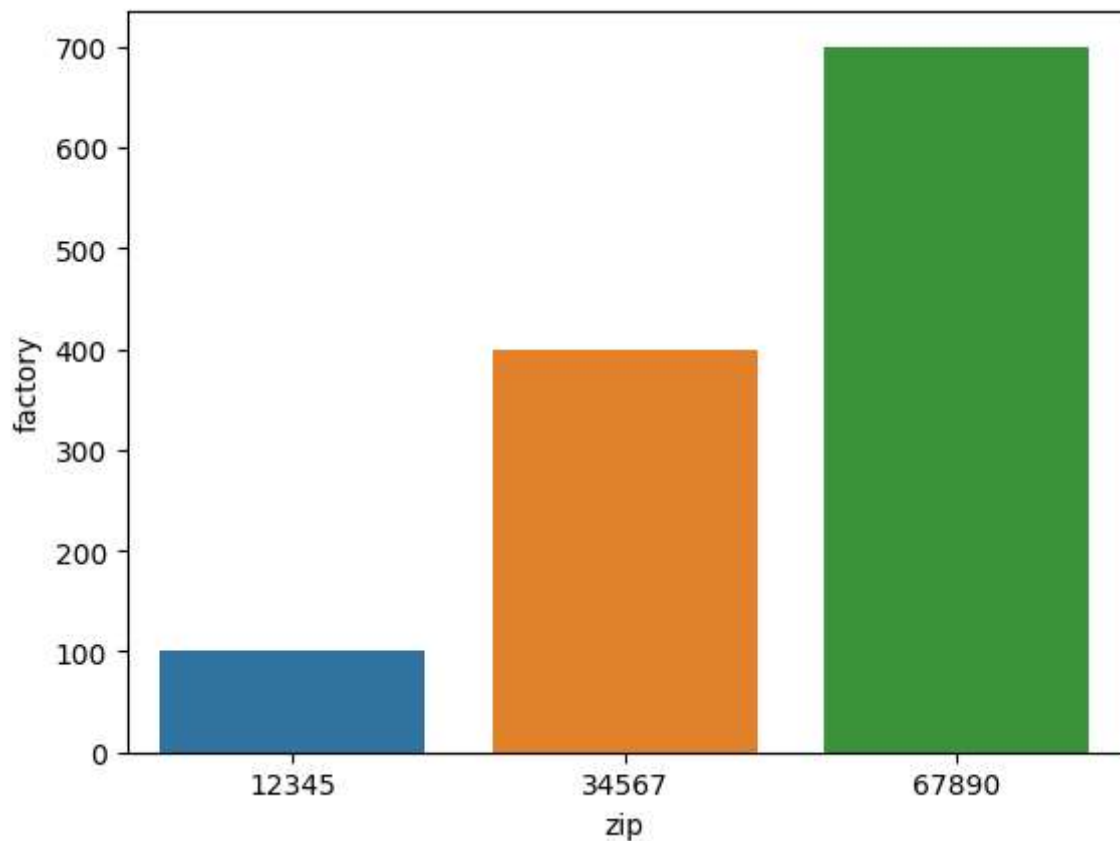
```
In [ ]: fasla
```

```
Out[ ]:
```

	zip	factory	warehouse	retail
0	12345	100	200	300
1	34567	400	500	600
2	67890	700	800	900

```
In [ ]: #plot
sns.barplot(x='zip',y='factory',data=fasla)
```

```
Out[ ]: <Axes: xlabel='zip', ylabel='factory'>
```



```
In [ ]: #convert Long format
        fasla_long=fasla.melt(id_vars='zip',var_name='location_type',value_name='distance')
        fasla_long
```

```
Out[ ]:
```

	zip	location_type	distance
0	12345	factory	100
1	34567	factory	400
2	67890	factory	700
3	12345	warehouse	200
4	34567	warehouse	500
5	67890	warehouse	800
6	12345	retail	300
7	34567	retail	600
8	67890	retail	900

```
In [ ]: import seaborn as sns
        sns.barplot(x='zip',y='distance', hue="location_type", data=fasla_long)
```

```
Out[ ]: <Axes: xlabel='zip', ylabel='distance'>
```

